PROJECT REPORT

CloudCostLens

An Intelligent AWS Architecture Advisor and Automated DevSecOps Provisioning Platform

Submitted By: -
Name:- Aadi Jain
Roll No:- 27
Registration No: - 12304968
Section :- 2OM58

Submitted to :- Mrs. Arshiya

Department of Computer Science and Engineering Lovely Professional University

Punjab, India

1. Abstract

The rapid adoption and migration to cloud computing platforms have introduced significant complexity in selecting cost-effective, scalable, and operationally efficient infrastructure. CloudCostLens is a comprehensive decision-support and automation platform designed to solve this challenge. By evaluating specific user constraints—such as financial budget, anticipated traffic volume, and operational effort preferences—the platform utilizes a deterministic rules engine to recommend optimized Amazon Web Services (AWS) architectures.

Beyond acting as an advisory tool, CloudCostLens features a programmatic Infrastructure as Code (IaC) engine that automatically provisions the selected architecture using Terraform without requiring manual AWS console interaction. To mitigate the risk of runaway cloud costs during testing and development, an automated background scheduler strictly enforces a two-hour resource lifespan, automatically executing a teardown of provisioned resources. The entire platform is engineered around a production-grade DevSecOps pipeline, ensuring continuous integration, multi-layered security scanning, and robust Kubernetes-based orchestration.

2. Introduction

In the modern software development landscape, developers and organizations frequently struggle with the "paradox of choice" when designing cloud infrastructure. Amazon Web Services (AWS) alone offers over 200 distinct services, each with highly specific pricing models, scaling behaviors, and maintenance requirements.

Traditionally, provisioning cloud infrastructure involves manual configuration via web consoles. This manual approach is highly error-prone, lacks proper version control, and often leads to security misconfigurations. Furthermore, experimental or temporary cloud resources are frequently forgotten by developers, resulting in "orphaned" resources that generate massive, unexpected billing charges.

CloudCostLens directly addresses these issues by providing a bridge between architectural planning and infrastructure execution. It empowers developers to make financially sound, data-driven architectural decisions while simultaneously abstracting away the steep learning curve of writing raw Terraform scripts, managing network VPCs, and configuring access policies manually.

3. Problem Statement

The development of CloudCostLens was motivated by three primary industry challenges:

1. Cognitive Overload and Decision Fatigue: Software engineers spend excessive time comparing AWS services, deciphering complex pricing calculators, and forecasting potential costs rather than focusing on building core application features.

2. Configuration Drift and Security Vulnerabilities: Manual AWS deployments lead to inconsistent environments between staging and production (configuration drift). Human error during manual network configuration frequently exposes sensitive ports and data, leading to severe security breaches.

3. Runaway Cloud Costs: In development environments, experimental infrastructure is often provisioned and subsequently forgotten. These idle resources continue to accrue charges, leading to financial waste.

4. Proposed Solution & Core Platform Features

CloudCostLens offers an automated, secure, and cost-aware approach to cloud infrastructure lifecycle management through the following core features:

Intelligent Recommendations: A proprietary, rule-based Java decision engine that scores AWS services against user parameters.

One-Click Provisioning Orchestration: Dynamically generates custom `terraform.tfvars` files and executes raw Terraform shell commands (`Init → Plan → Apply → Destroy`) natively from the backend application.

Auto-Destroy Safety Net: A scheduled background thread (using Spring's `ThreadPoolTaskScheduler`) that automatically initiates a complete teardown of provisioned AWS resources after exactly 120 minutes to prevent unintended billing.

Proactive Budget Guardrails: The system mathematically validates proposed architectural costs against user-defined financial limits before allowing any deployment execution.

Secure Access Control: JSON Web Token (JWT) based authentication ensuring strict role-based access control (RBAC) over cloud mutation operations.

5. System Architecture and Modular Design

The application is engineered using a containerized, domain-aligned **Modular Monolith** architecture. This ensures high cohesion within modules and loose coupling across the system, making the codebase highly maintainable.

5.1 Presentation Layer (Frontend)

The frontend is built using React 18 and Vite, utilizing a custom CSS3 glassmorphism design system. It operates as a Single Page Application (SPA) that provides interactive forms for requirements gathering, visualizes complex infrastructure topologies, and streams real-time console logs from the backend during active deployments.

5.2 Business Logic Layer (Spring Boot Backend)

The backend is developed in Java 21 using Spring Boot 3.2. It enforces strict module boundaries to isolate domains:

`auth`: Manages user login, registration, and JWT token issuance.

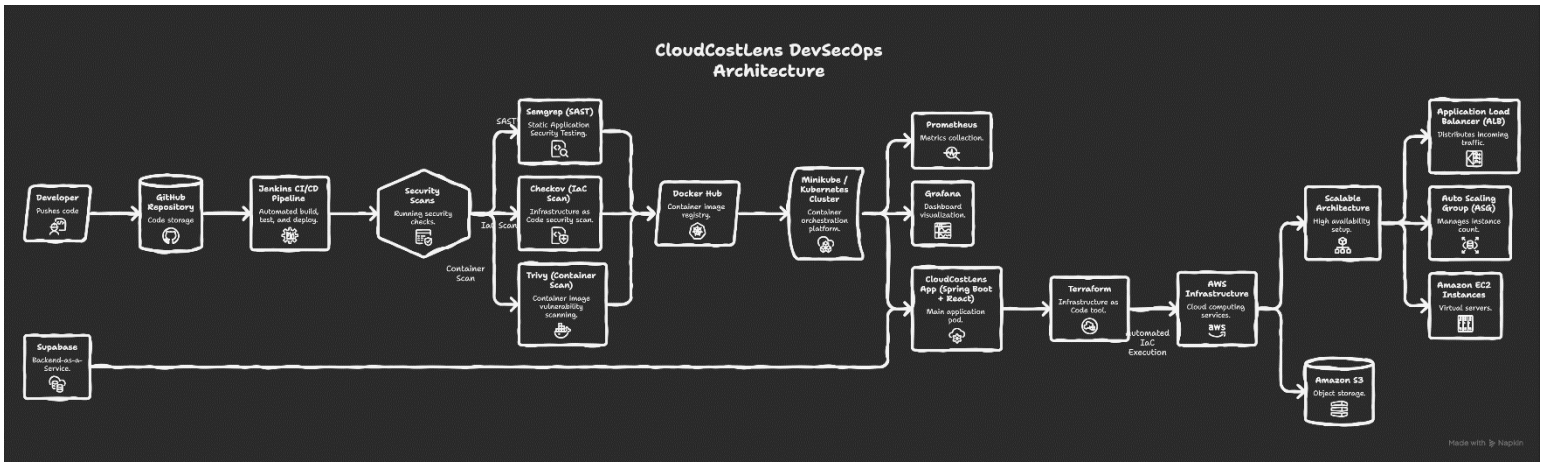
`recommendation`: Contains the mathematical scoring algorithms, pricing normalization logic, and the AWS service knowledge base.

`deployment`: Houses the Terraform orchestration logic, local file system management for workspaces, and process stream capturing.

`config`: Manages global application properties, Cross-Origin Resource Sharing (CORS) configurations, security filter chains, and global exception handling.

5.3 Persistence Layer

The platform utilizes Supabase (PostgreSQL) as a cloud-hosted relational database. It is responsible for securely storing user account profiles, tracking deployment audit histories, and maintaining the state of past architectural recommendations.



6. The Intelligent Decision Engine Logic

At the heart of the platform is the `DecisionEngineService`. Rather than relying on simple static mapping, the engine utilizes a deterministic scoring formula to rank eligible AWS services. The weighted scoring algorithm prioritizes financial constraints while ensuring performance metrics are met:

* Cost Match (40% Weight): Evaluates whether the service's base cost and dynamic scaling cost fit within the user's hard budget limit.

* Scalability Match (30% Weight): Aligns the infrastructure's maximum throughput capabilities with the user's anticipated traffic volume (Low, Medium, High).

* Operational Effort (20% Weight): Matches the user's desire for control. If a user requests "low effort," the system heavily penalizes unmanaged services (like raw EC2) in favor of fully managed serverless options (like AWS Lambda).

* Use-Case Fit (10% Weight): Provides a slight preference boost to services explicitly optimized for the requested application type (e.g., Static Websites vs. Event-Driven APIs).

7. Automated Cloud Provisioning (IaC)

CloudCostLens translates theoretical recommendations into physical infrastructure using HashiCorp Terraform.

7.1 ProcessBuilder Orchestration

Instead of relying on external CI servers to run Terraform, the Java backend utilizes `java.lang.ProcessBuilder` to execute Terraform CLI commands directly within isolated local workspace directories. The system dynamically generates environment variables and injects AWS credentials securely at runtime, completely bypassing the need to store credentials on disk.

7.2 Supported Topologies

The engine maps user requirements to one of three dynamically parameterized Terraform modules:

1. Scalable App Module (`scalable_app`): Designed for high-traffic, enterprise workloads. It deploys an Application Load Balancer (ALB) connected to an Auto Scaling Group (ASG) utilizing dynamic EC2 Launch Templates spanning multiple Availability Zones.

2. Web App Module (`web_app`): Tailored for low-traffic MVPs. It provisions a standalone EC2 instance with an Elastic IP and a strictly locked-down ingress Security Group.

3. Storage App Module (`storage_app`): Engineered for static asset delivery. It configures a private Amazon S3 bucket masked behind an Amazon CloudFront CDN utilizing Origin Access Control (OAC) for maximum security.

8. DevSecOps and Security Pipeline

The project implements a rigorous "Shift-Left" security philosophy, enforced by an automated Jenkins CI/CD Pipeline. Every code commit must pass through multiple security gates before deployment:

1. SAST (Static Application Security Testing): Semgrep analyzes the raw Java and JavaScript source code to identify coding anti-patterns, hardcoded secrets, and logical vulnerabilities.

2. SCA (Software Composition Analysis): OWASP Dependency-Check and npm audit scan third-party libraries and dependencies to ensure no packages with known Common Vulnerabilities and Exposures (CVEs) are introduced.

3. IaC Security Scanning: Checkov statically analyzes the raw Terraform modules to detect cloud misconfigurations (e.g., ensuring S3 buckets are not publicly readable and encryption is enabled).

4. Container Scanning:Trivy inspects the compiled Docker image layers to identify outdated operating system packages and vulnerabilities within the base alpine Linux images.

5. DAST (Dynamic Application Security Testing): Post-deployment, OWASP ZAP executes active penetration testing techniques against the live application endpoints to hunt for runtime flaws such as Cross-Site Scripting (XSS) and SQL Injection.

9. Deployment and Orchestration Infrastructure

The execution environment of CloudCostLens is highly resilient, leveraging modern orchestration and configuration management tools.

9.1 Containerization and Kubernetes Orchestration

The application stack is fully containerized using multi-stage Dockerfiles. In production environments, the containers are managed by Kubernetes (locally simulated via Minikube).

* Deployments: Ensure high availability by managing multi-replica Pods and enabling zero-downtime rolling updates.

* Services: Abstract internal networking, exposing the backend via ClusterIP and the frontend via NodePort.

* ConfigMaps and Secrets: Application configurations are completely decoupled from the codebase. Non-sensitive variables are stored in ConfigMaps, while JWT keys and database passwords are encrypted within Kubernetes Secrets.

* Persistent Volume Claims (PVC): Guarantee that critical state data (such as local Terraform state files and audit logs) survive Pod restarts or node failures.

9.2 Configuration Management with Ansible

The underlying host servers running the Kubernetes cluster and CI/CD tools are provisioned using Ansible. Automated Playbooks execute role-based configurations: updating core OS packages (`common` role), installing the Docker runtime engine (`docker` role), and configuring strict Uncomplicated Firewall (UFW) rules to harden the server perimeter (`security` role).

10. Monitoring and Observability

To ensure continuous reliability, the platform utilizes a robust observability stack. The Spring Boot backend integrates the Micrometer library and Spring Boot Actuator to expose deep JVM telemetry.

Prometheus scrapes these metrics every 15 seconds from within the Kubernetes cluster. Finally, Grafana connects to Prometheus to visualize the data on live dashboards, tracking critical health indicators such as HTTP request latency, CPU utilization, garbage collection pauses, and HikariCP database connection pool saturation.

11. Performance Engineering and Load Testing

To validate the architectural stability of the platform, structured load testing was conducted using the k6 performance testing framework.

* Concurrent Load Test (Standard Traffic): The system successfully sustained a throughput of ~1,216 requests per second with 1,000 concurrent Virtual Users (VUs) over a sustained period, maintaining an average response latency of ~791ms and a failure rate below 1.4%.

* Ramp-Up Stress Test (Extreme Traffic): By aggressively ramping up to 3,000 concurrent VUs, the testing successfully identified the system's breaking point. The bottleneck analysis revealed that the single relational database connection pool became saturated, resulting in request timeouts.

12. Conclusion

CloudCostLens successfully bridges the complex gap between theoretical architectural planning and physical cloud execution. By mathematically quantifying infrastructure tradeoffs and integrating programmatic Terraform execution, it eliminates manual provisioning errors and significantly reduces "time-to-cloud" for developers. Furthermore, the implementation of a comprehensive DevSecOps pipeline, multi-layered security scanning, and Kubernetes orchestration demonstrates a robust, production-ready implementation that prioritizes security, scalability, and financial cost-efficiency.

13. Future Scope

While the current platform is highly capable, the roadmap for future enhancements includes:

* Caching Layer Integration: Implementing a Redis cluster to cache frequently accessed recommendation models and static AWS pricing data, thereby reducing the load on the primary PostgreSQL database during high-traffic spikes.

* Horizontal Pod Autoscaling (HPA): Configuring Kubernetes HPA to dynamically scale the number of Spring Boot backend Pods based on real-time CPU and memory utilization metrics scraped by Prometheus.

* Multi-Cloud Capabilities: Expanding the decision engine's knowledge base and the Terraform IaC modules to support cross-cloud provisioning across Microsoft Azure and Google Cloud Platform (GCP), offering true cloud-agnostic recommendations.